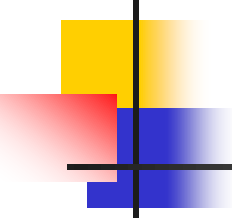


Interfaces graphiques

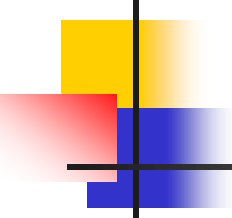


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



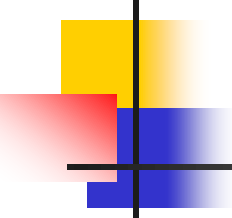
Plan du cours

- Introduction
- Fenêtres et boîtes de dialogues
- Conteneurs intermédiaires
- Composants de base
- Gestionnaire de dispositions



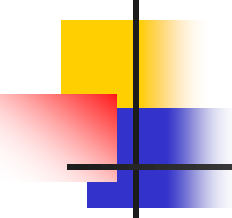
Introduction

- **Interface graphique** ou *Graphical User Interface* (**GUI**)
 - façade du programme qui le lie avec l'extérieur
 - la plupart des logiciels disposent d'une GUI
- Des composants Java pour réaliser des GUI
- Trois bibliothèques principalement utilisées
 - **AWT** (*Abstract Window Toolkit*) (**java.awt.***)
 - **Swing** (**javax.swing.***)
 - repose sur **AWT** qu'il a remplacé progressivement
 - plus performant
 - **JavaFX**
 - la référence depuis Java 8, inspiré du Web



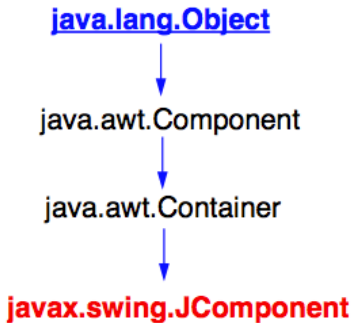
Pourquoi Swing?

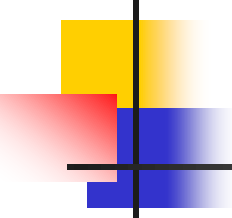
- Bibliothèque riche de classes graphiques
- Un ensemble de composants graphiques légers
- Permet de réaliser des applications graphiques complexes et portables
- Il est simple et multiplateforme
- Il utilise l'architecture MVC
- Il a de bonnes performances



Composants Swing

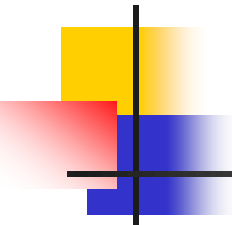
- Une GUI est constituée de composants graphiques imbriqués les uns aux autres
- Les composants sont généralement imbriqués en 3 niveaux
 - Conteneurs de niveau supérieur (fenêtre)
 - contenir des composants des 2 autres niveaux (JFrame)
 - Conteneurs intermédiaires
 - regrouper en leurs seins des composants atomiques (JPanel)
 - Composants de base
 - boutons, zones de saisie, menus, listes déroulantes, etc.





Conteneurs de niveau supérieur

- Principales fenêtres de Swing
 - JFrame
 - JDialog
 - JOptionPane
 - JFileChooser
 - JColorChooser



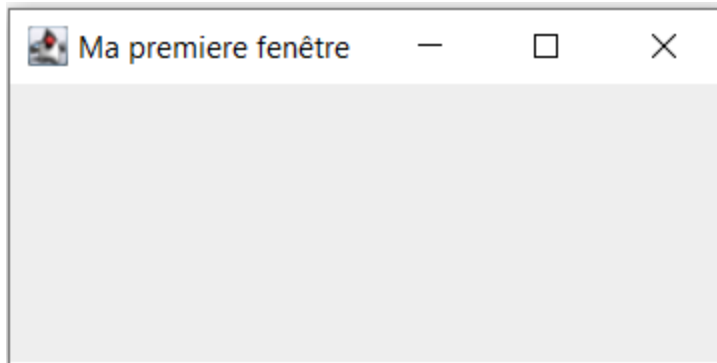
JFrame

- **JFrame** permet de créer une **fenêtre principale**
 - composant graphique fondamental
 - avec une barre de titre, un cadre et une barre de menu
 - peut être agrandie, diminuée ou redimensionnée
- Méthodes courantes de la classe **JFrame**
 - **setSize(larg, haut)**: taille de la fenêtre
 - **larg** : taille horizontale en pixels
 - **haut** : taille verticale en pixels
 - **setTitle("Ma première fenêtre")**: titre de la fenêtre
 - **setVisible(boolean)**: **true** pour visible et **false** pour caché
 - **setLocation(x, y)**: position de la fenêtre (coin supérieur gauche)
 - **pack()**: taille minimale pour afficher les composants

JFrame

■ Exemple

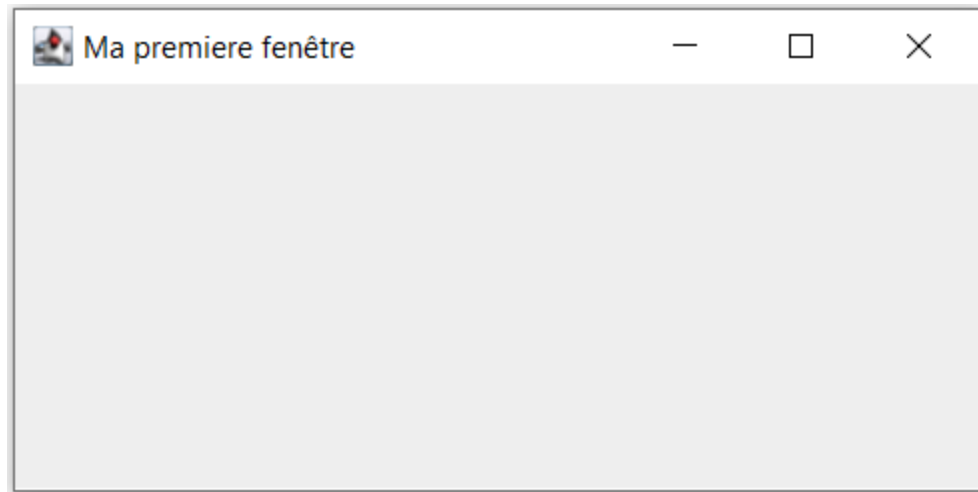
```
package sn.uasz.m1.gui;
import javax.swing.* ;
public class MaFenetre {
    public static void main(String[] args) {
        JFrame fen = new JFrame() ;
        fen.setSize(300, 150) ;
        fen.setTitle("Ma premiere fenêtre") ;
        fen.setVisible(true) ; // pas visible par défaut
    }
}
```

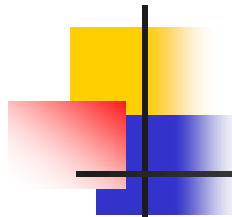


JFrame

- On peut aussi étendre la classe JFrame

```
package sn.uasz.m1.gui;
import javax.swing.* ;
public class MaFenetre extends JFrame{
    public MaFenetre (){
        setTitle("Ma premiere fenetre") ;
        setSize(400, 200) ;
    }
    public static void main(String[] args) {
        JFrame fenetre = new MaFenetre() ; // créer une fenetre
        fenetre.setVisible(true) ; // rendre visible la fenetre
    }
}
```



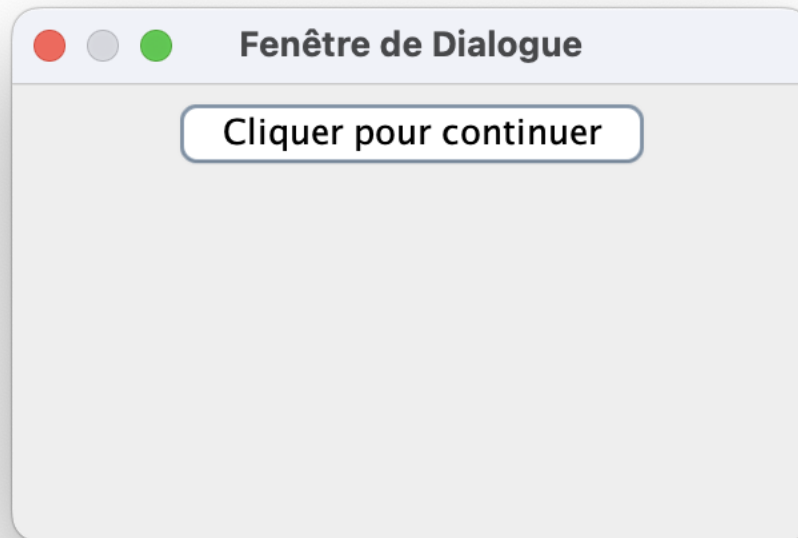


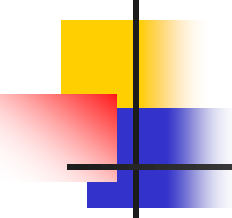
JFrame

- Autres méthodes usuelles de la classe **JFrame**
 - **add(JComponent)**: ajout d'un composant graphique
 - **setBounds(x, y, larg, haut)**: position et taille
 - **setLocationRelativeTo(null)**: centrer la fenêtre
 - **setDefaultCloseOperation(JFrame.xxx)**: impose un comportement lors de la fermeture de la fenêtre
 - **JFrame.DISPOSE_ON_CLOSE** // ferme la fenêtre
 - **JFrame.EXIT_ON_CLOSE** // ferme l'application
 - **JFrame.DO_NOTHING_ON_CLOSE** // ne rien faire
 - **setResizable(boolean)**: **false** pour empêcher le redimensionnement de la fenêtre
 - **setLayout(LayoutManager)**: gestion de la disposition des composants

JDialog

- Fenêtre secondaire dépendante d'une fenêtre principale
- Elle apparaît au dessus d'une autre fenêtre
- Elle est souvent temporaire et **modale**
 - pas d'interaction avec la fenêtre mère tant que la boîte de dialogue est ouverte



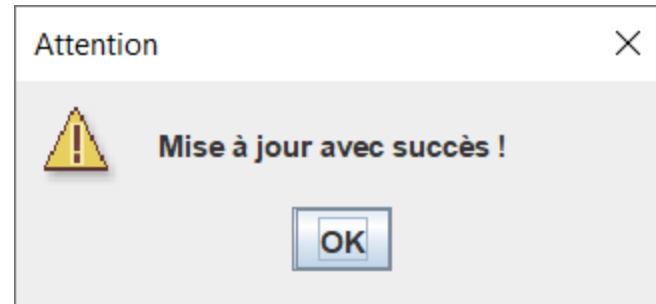
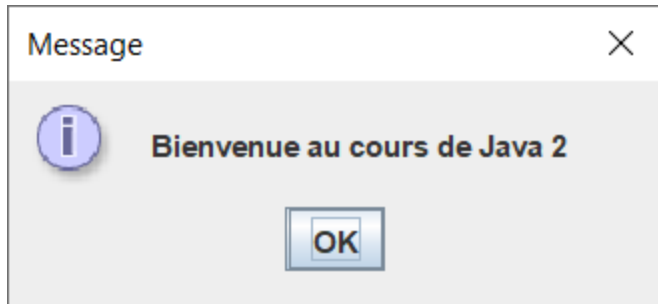


JOptionPane

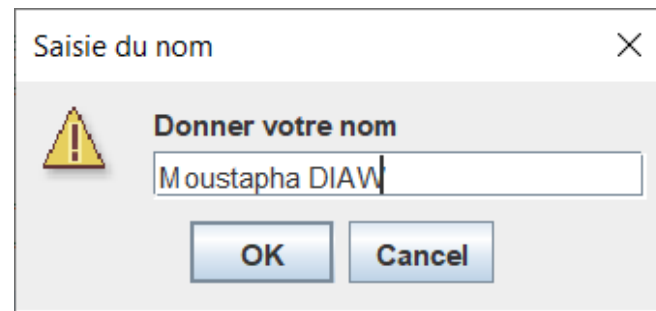
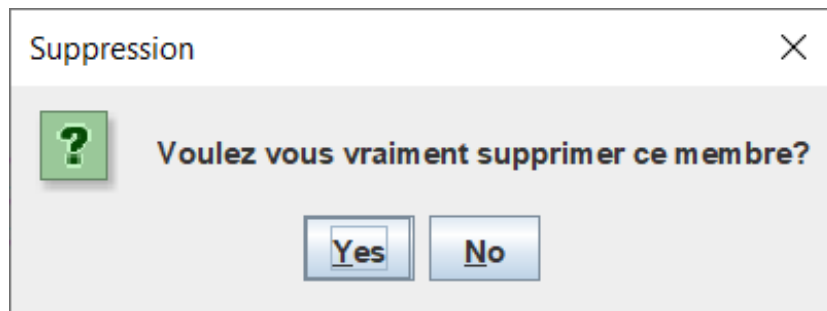
- Cette classe fournit des boîtes de dialogue standards
- Elle sert à afficher des informations ou recueillir des données de l'utilisateur
- Trois types de boîtes de dialogue
 - Message d'information
 - `JOptionPane.showMessageDialog()`
 - Boîte de dialogue de confirmation (avec `Yes`, `No` et `Cancel`)
 - `JOptionPane.showConfirmDialog()`
 - Boîte de dialogue de saisie de données
 - `JOptionPane.showInputDialog()` (avec `OK` et `Cancel`)

JOptionPane

- Exemples avec `showMessageDialog()`



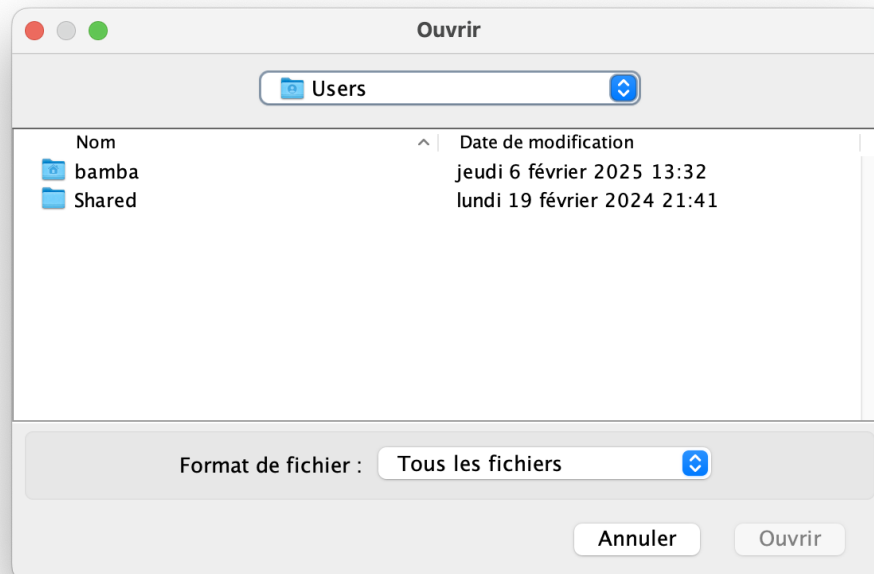
- Exemples avec `showConfirmDialog` et `showInputDialog()`

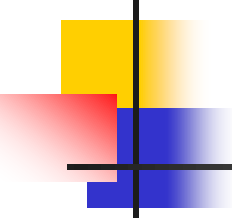


JFileChooser

- Elle permet de sélectionner un fichier en parcourant l'arborescence du système de fichier
- Exemple

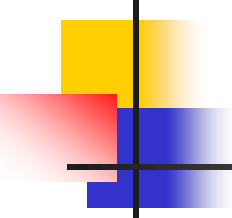
```
JFileChooser fc = new JFileChooser();  
int returnVal = fc.showOpenDialog(fenetre);  
if (returnVal == JFileChooser.APPROVE_OPTION){  
    File file = fc.getSelectedFile();  
}
```





Conteneurs intermédiaires

- Un conteneur intermédiaire permet de regrouper et d'organiser d'autres composants graphiques
- Les conteneurs intermédiaires peuvent être emboîtés
- Ils jouent un rôle fondamental dans la conception de GUI
- Ils permettent de placer des composants dans une fenêtre
- La classe `JFrame` dispose d'une méthode `getContentPane()` renvoyant un conteneur (`Container`)



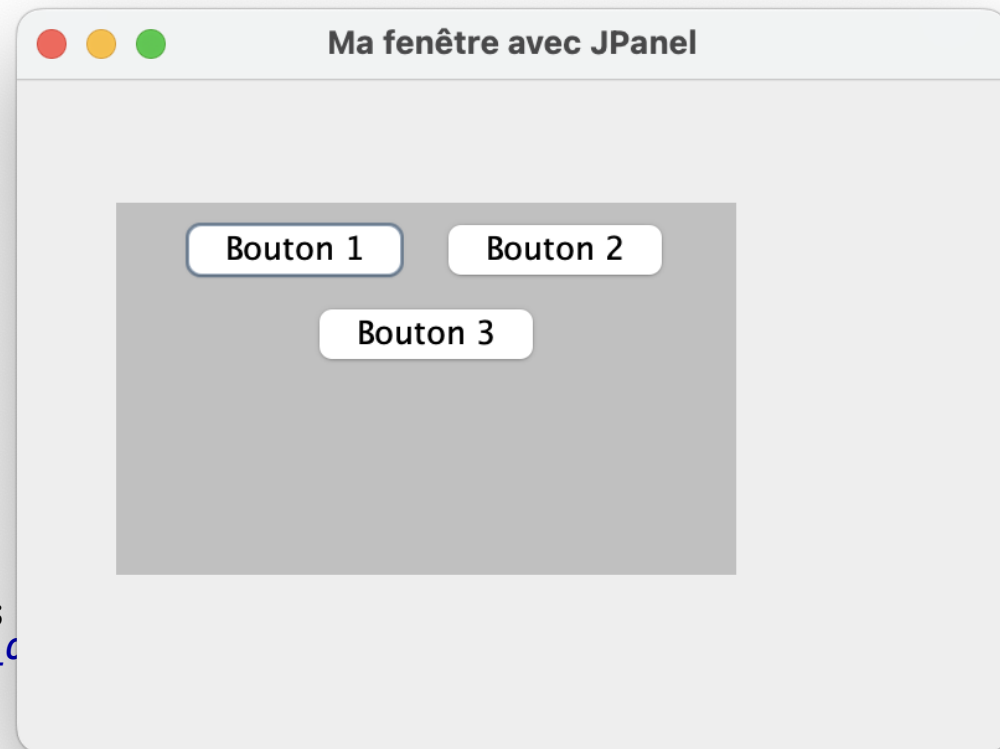
Conteneurs intermédiaires

- Méthodes communément utilisées
 - `add(JComponent)`: ajout d'un composant
 - `setSize(long, larg)`: fixe la taille du conteneur
 - `setBounds(x, y, long, larg)`: positionne le conteneur et fixe la taille
 - `setLayout(LayoutManager)`: gère la disposition des composants du conteneur
 - `setVisible(true)`: rend le conteneur visible
- Quelques conteneurs intermédiaires courants
 - `JPanel`
 - `JScrollPane`
 - `JTabbedPane`
 - `JSplitPane`
 - ...

JPanel

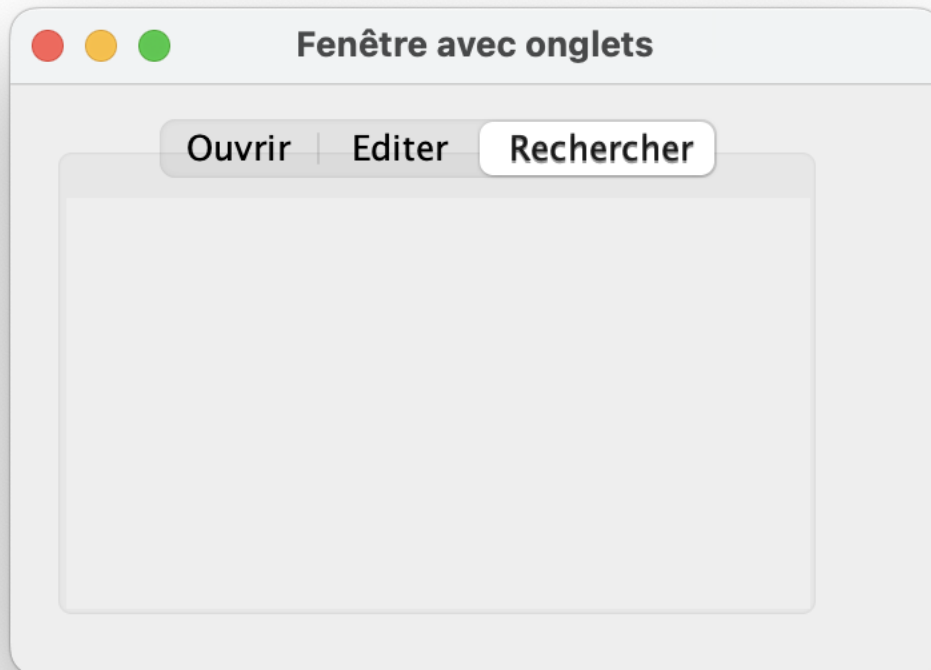
- **JPanel** représente un panneau sans représentation graphique
- Exemple

```
public static void main(String[] args) {  
    JFrame fen = new JFrame() ;  
    JPanel p = new JPanel();  
    JButton b1 = new JButton("Bouton 1");  
    JButton b2 = new JButton("Bouton 2");  
    JButton b3 = new JButton("Bouton 3");  
    p.add(b1);  
    p.add(b2);  
    p.add(b3);  
    p.setBounds(40,50,250,150);  
    p.setBackground(Color.lightGray);  
    fen.add(p);  
    fen.setSize (400, 300) ;  
    fen.setTitle ("Ma fenêtre avec JPanel") ;  
    fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    fen.setLayout(null);  
    fen.setVisible(true) ;  
}
```



JScrollPane et JTabbedPane

- **JScrollPane** permet d'afficher une partie d'un composant trop grand en utilisant des barres de défilement
- **JTabbedPane** représente un panneau composé d'un certain nombre d'**onglets** affichant chacun un composant

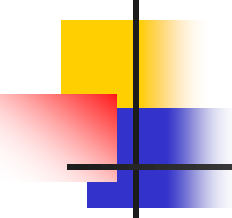


JSplitPane

■ JSplitPane

- un panneau subdivisé en deux composants redimensionnables
- les deux composants peuvent être alignés de gauche à droite ou de haut à bas

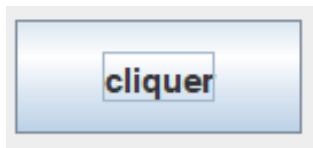
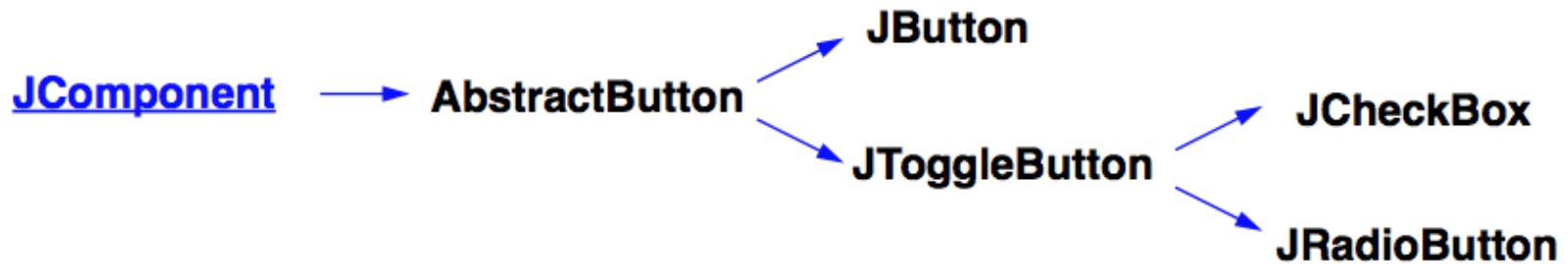




Composants de base

- Ces composants héritent de la classe `JComponent`
- Ils sont constitués de
 - Boutons: `Jbutton`, `JRadioButton`
 - Cases à cocher: `JCheckBox`
 - Etiquettes: `JLabel`
 - Champs de textes: `JTextField`, `JPasswordField`
 - Listes de choix: `JList`, `JComboBox`
 - Tableaux: `JTable`
 - ...

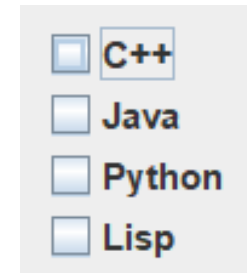
Boutons



JButton



JRadioButton
Choix exclusif

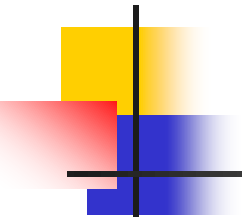


JCheckBox
Choix indépendants

Boutons radio (JRadioButton)

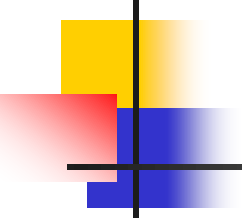
```
package sn.uasz.m1.gui;
import javax.swing.* ;
public class Boutons {
    public static void main(String[] args) {
        JFrame f=new JFrame("Exemple Boutons Radio");
        Image icone = Toolkit.getDefaultToolkit().getImage("C:/java2.png");
        f.setIconImage(icone);
        b.setBounds(50,100,100, 40);
        JRadioButton r1=new JRadioButton("Homme");
        JRadioButton r2=new JRadioButton("Femme");
        r1.setBounds(75,100,100,30);
        r2.setBounds(75,130,100,30);
        ButtonGroup bg=new ButtonGroup();
        bg.add(r1);
        bg.add(r2);
        f.add(r1);f.add(r2);
        f.setSize(400,200);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```





Boutons radio (JRadioButton)

- Création de bouton radio
 - `JRadioButton r1 = new JRadioButton("Homme");`
 - `JRadioButton r1 = new JRadioButton("Femme", true);` //coché par défaut
- Connaître l'état d'un bouton radio
 - `boolean coche = r1.isSelected();`
- Forcer l'état d'un bouton radio
 - `r1.setSelected(true);`
- Regrouper les boutons radios
 - `JButtonRadio r1 = new JButtonRadio("Homme");`
 - `JButtonRadio r2 = new JButtonRadio("Femme");`
 - `ButtonGroup g = new ButtonGroup();`
 - `g.add(r1); g.add(r2);`



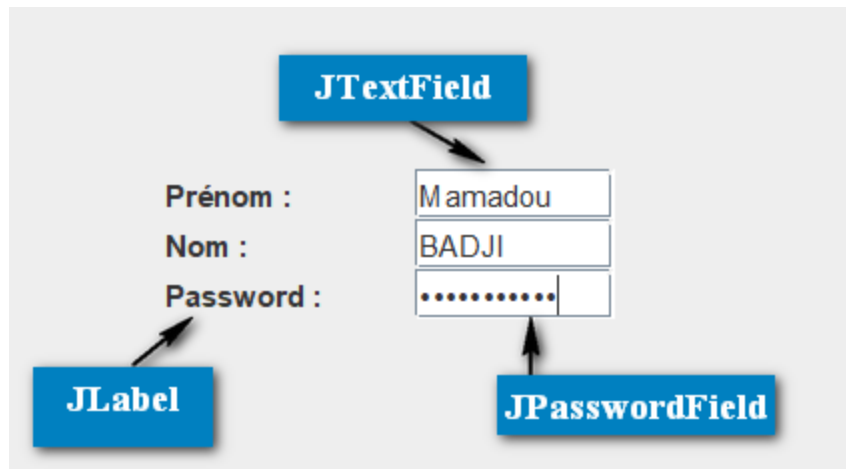
Cases à cocher (JCheckBox)

- Création de case à cocher
 - `JCheckBox case = new JCheckBox("Java");`
 - `JCheckBox case = new JCheckBox("Java", true);` //coché par défaut
- Connaître l'état d'une case
 - `boolean coche = case.isSelected();`
- Forcer l'état d'une case
 - `case.setSelected(true);`

Composants de saisie de textes

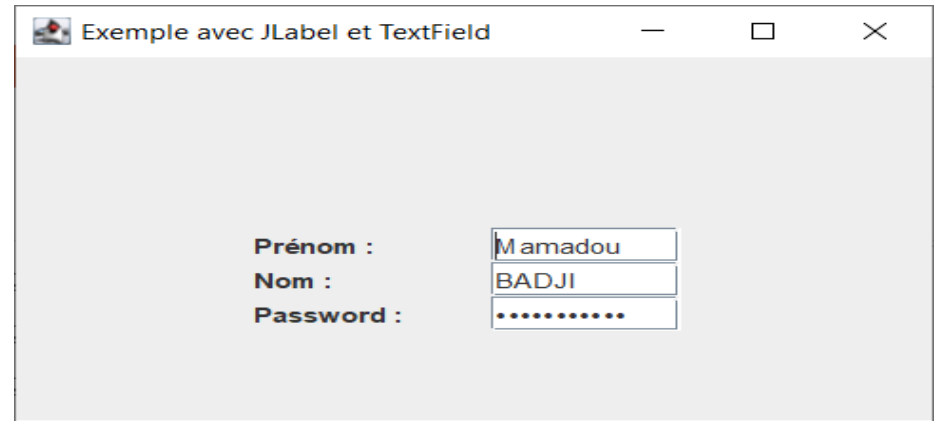


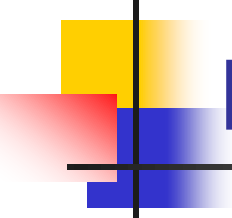
- **JTextField** composé d'une seule ligne (contrairement au **JTextArea**)
- **JPasswordField** permet la saisie de mots de passe
- **JLabel** permet d'afficher un texte ou une image



Composants de saisie de textes

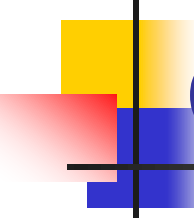
```
package sn.uasz.m1.gui;
import javax.swing.*;
public class Textex {
    public static void main(String[] args) {
        JFrame f= new JFrame("Exemple avec JLabel et TextField");
        JLabel label1=new JLabel("Prénom : ");
        JLabel label2=new JLabel("Nom : ");
        JTextField t1=new JTextField("Mamadou");
        JTextField t2=new JTextField("BADJI");
        label1.setBounds(100,100,80,20);
        t1.setBounds(200,100, 80,20);
        label2.setBounds(100,120, 80,20);
        t2.setBounds(200,120,80,20);
        f.add(label1);
        f.add(t1);
        f.add(label2);
        f.add(t2);
        JLabel label3=new JLabel("Password :");
        JPasswordField mdp = new JPasswordField("khadimdrame");
        label3.setBounds(100,140, 80,20);
        mdp.setBounds(200,140,80,20);
        f.add(mdp);
        f.add(label3);
        f.setSize(400,250);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```





Étiquettes (JLabel)

- Création d'une étiquette
 - `JLabel label = new JLabel();`
 - `JLabel label = new JLabel("mon label");`
- Modification de l'étiquette
 - `label.setText("nouveau label");`
- Modification de l'icône de l'étiquette
 - `label.setText(Icon icon);`

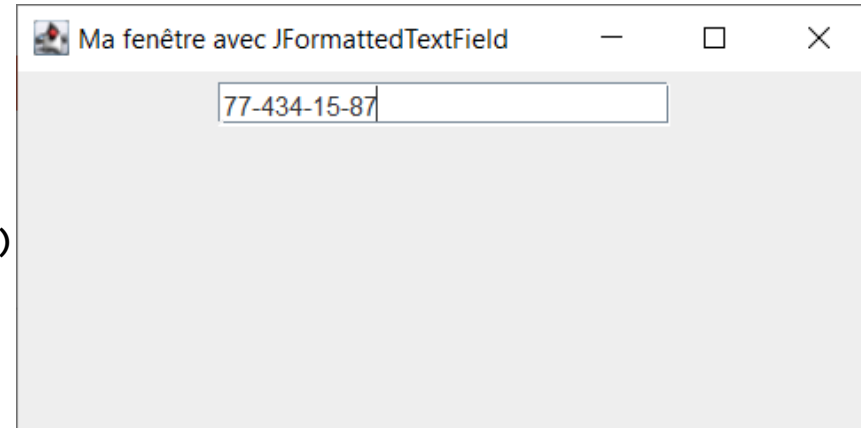


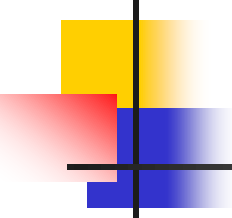
Champs de textes (JTextField)

- Création d'un champs de texte
 - `JTextField field = new JTextField(30);`
 - `JTextField field = new JTextField("texte par défaut");`
 - `JTextField field = new JTextField("texte par défaut", 30);`
- Rendre un champs de texte éditable ou non
 - `field.setEditable(false);` // champ non modifiable
 - `field.setEditable(true);` // champ modifiable
- Récupérer la valeur d'un champs de texte
 - `String s = field.getText();`
- Mettre à jour la valeur d'un champs de texte
 - `field.setText("nouveau texte");`

JFormattedTextField

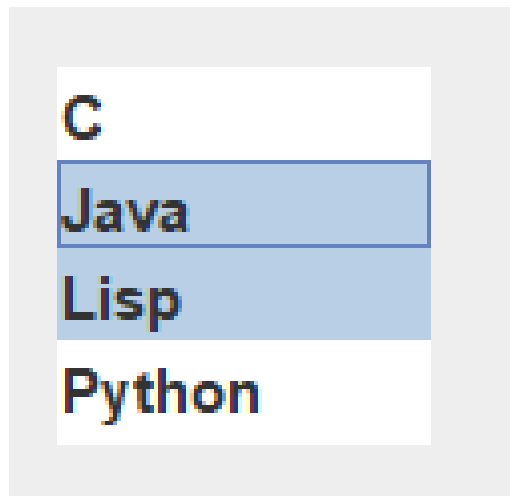
```
public static void main(String[] args) {
    JFrame fen = new JFrame() ;
    JPanel p = new JPanel();
    JFormattedTextField text = null;
    try {
        MaskFormatter formatter = new MaskFormatter("##-###-##-##");
        formatter.setPlaceholderCharacter('#');
        text = new JFormattedTextField(formatter);
        text.setColumns(20);
    } catch (Exception e) {
        e.printStackTrace();
    }
    p.add(text);
    fen.add(p);
    fen.setSize (400, 200) ;
    fen.setTitle ("Ma fenêtre avec JFormattedTextField")
    fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    fen.setVisible(true) ;
}
```

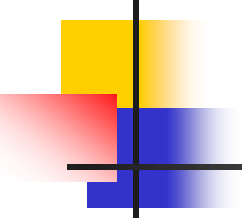




Liste de choix (JList)

- **JList** (liste de choix)
 - propose une liste d'éléments à choisir, en colonne
 - on peut sélectionner un ou plusieurs éléments
 - souvent contenue dans un **JScrollPane**





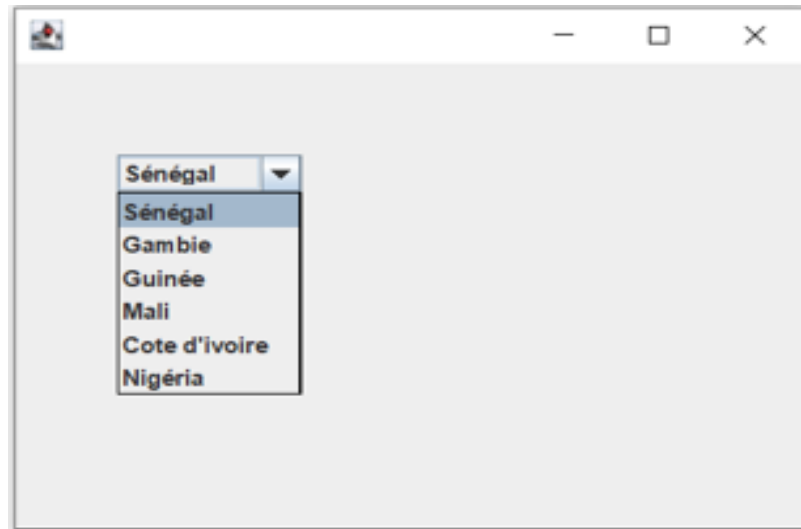
Liste de choix (JList)

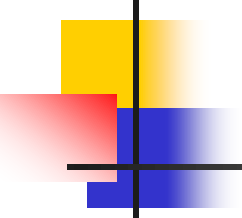
- Créer une liste de choix
 - `String [] langages = {"Java", "C", "Python", "Lisp", "C#"}`;
 - `JList liste = new JList(langages)`;
- Sélectionner un élément par défaut
 - `liste.setSelectedIndex(4)`;
- Fixer le nombre de lignes à afficher
 - `liste.setVisibleRowCount(5)`;
- Accéder aux éléments sélectionnés
 - `List valeurs = liste.getSelectedValuesList()`;

Liste de choix (JComboBox)

- **JComboBox** (liste déroulante)
 - propose un menu avec une liste de choix
 - supporte la sélection unique

```
JFrame f = new JFrame();  
String pays[]={ "Sénégal", "Gambie", "Guinée", "Mali", "Cote d'ivoire", "Nigéria"};  
JComboBox<String> cb=new JComboBox<String>(pays);  
cb.setBounds(50, 50,90,20);  
f.add(cb);  
f.setLayout(null);  
f.setSize(400,200);  
f.setVisible(true);
```





Liste de choix (JComboBox)

- Créer une liste de choix
 - `String[] langages = {"Java", "C", "Python", "Lisp", "C#" };`
 - `JComboBox cb = new JComboBox(langages);`
- Sélectionner un élément par défaut
 - `cb.setSelectedIndex(2); cb.setSelectedItem(Object);`
- Fixer le nombre d'éléments visibles
 - `cb.setMaximumRowCount(5);`
- Ajouter/supprimer un élément
 - `cb.addItem("Caml"); // cb.removeItem("Java");`
- Accéder à l'élément sélectionné
 - `Object valeur = cb.getSelectedItem();`

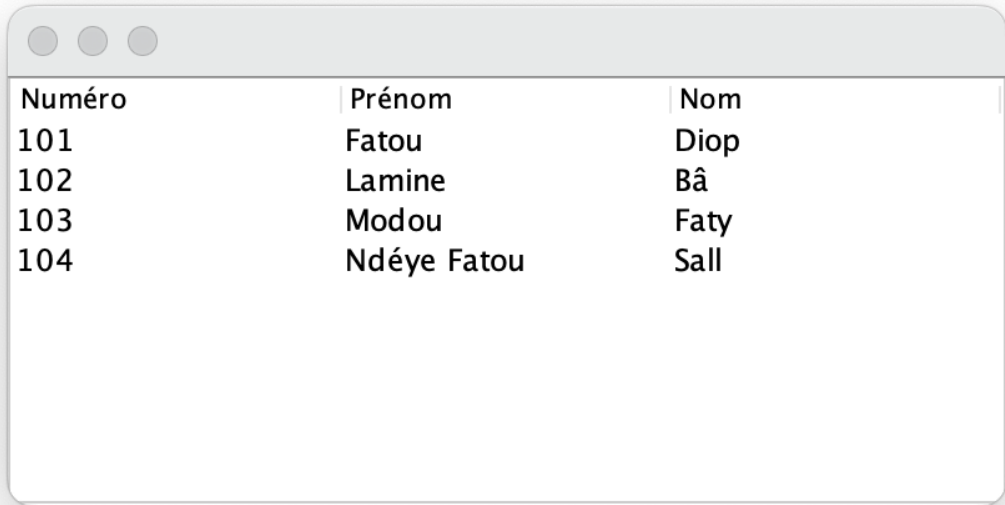
JTable

■ JTable

- utilisé pour afficher des données sous format tabulaire
- constitué de lignes et de colonnes

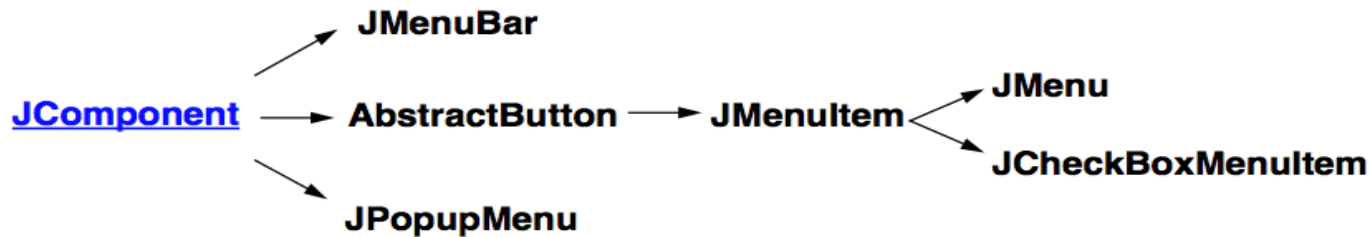
■ Exemple

```
JFrame f=new JFrame();
String data[][]={{ "101", "Fatou", "Diop"}, {"102", "Lamine", "Bâ"},
                  {"103", "Modou", "Faty"}, {"104", "Ndéye Fatou", "Sall"} };
String column[]={"Numéro", "Prénom", "Nom"};
JTable t=new JTable(data,column);
t.setBounds(30,40,200,300);
JScrollPane sp=new JScrollPane(t);
f.add(sp);
f.setSize(400,200);
f.setVisible(true);
```



Numéro	Prénom	Nom
101	Fatou	Diop
102	Lamine	Bâ
103	Modou	Faty
104	Ndéye Fatou	Sall

Menus

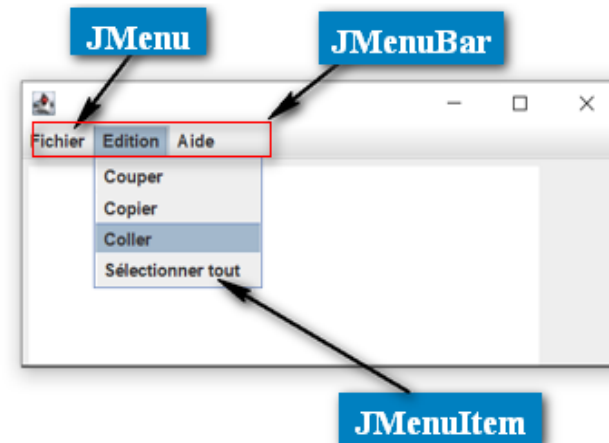
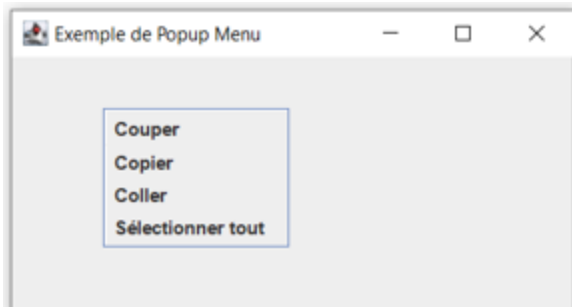


■ JMenuBar

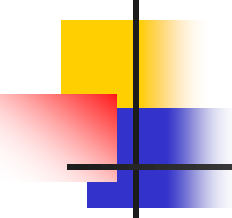
- barre de menus
- peut avoir plusieurs menus (**JMenu**)

■ JPopupMenu

- menu contextuel



■ JCheckBoxMenuItem

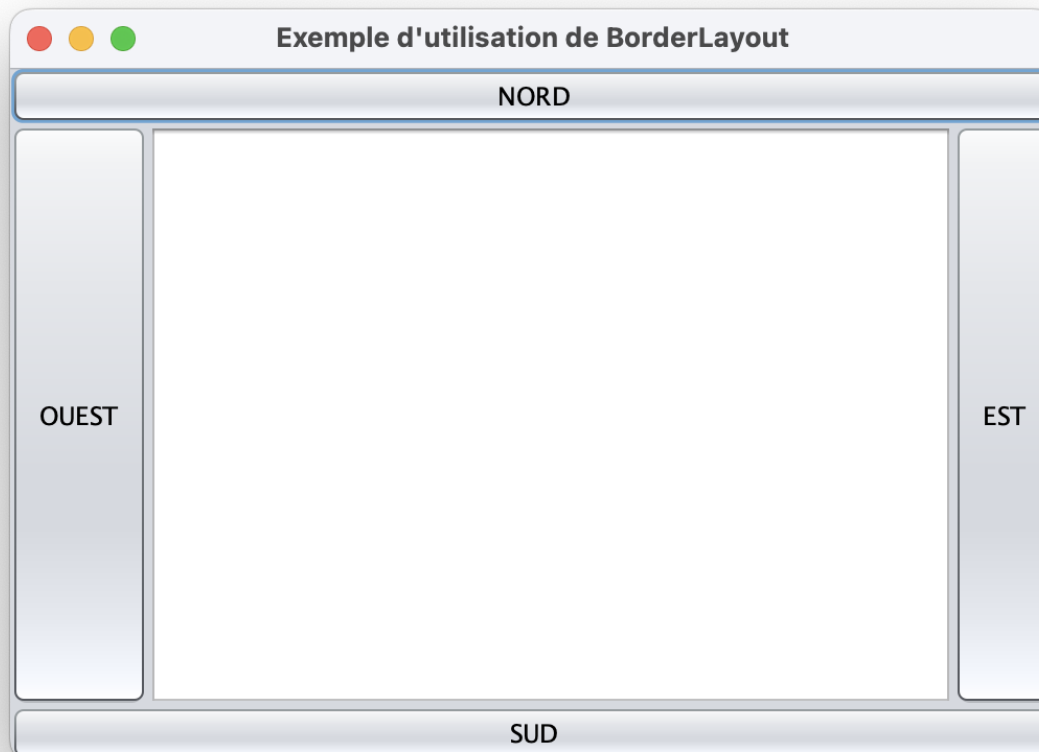


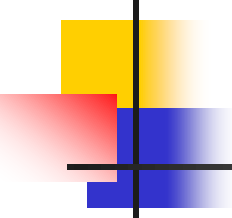
Layout managers

- Un *Layout manager* permet de gérer la disposition spatiale des composants dans un conteneur
- Il existe plusieurs types de **Layout managers**
 - BorderLayout
 - FlowLayout
 - GridLayout
 - CardLayout
 - GridBagLayout
 - BoxLayout
 - SpringLayout
 - etc.

BorderLayout

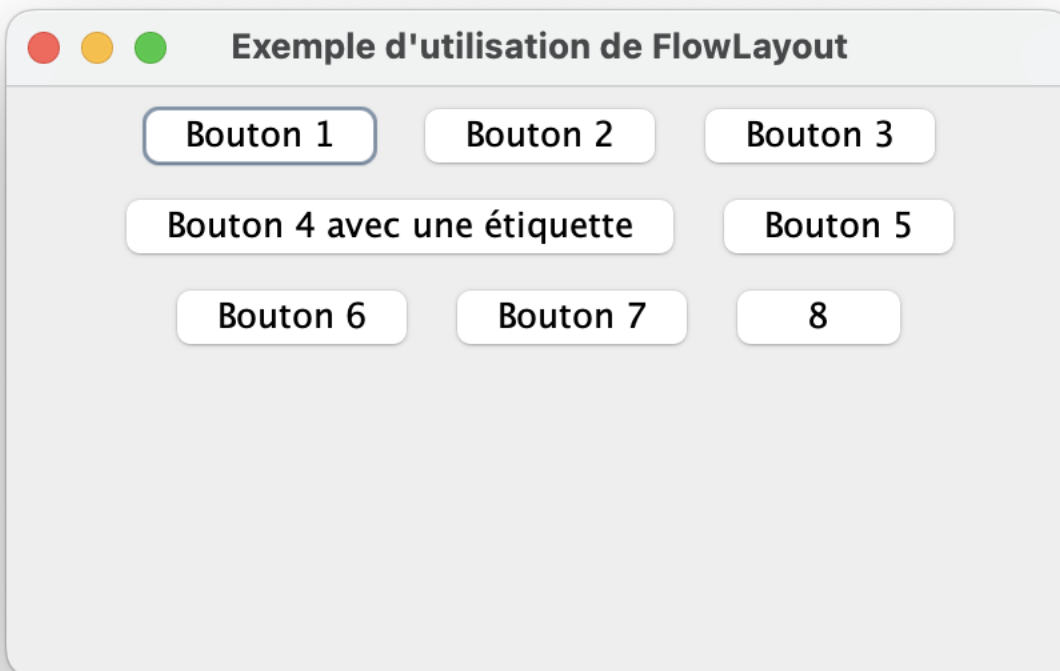
- **BorderLayout** permet d'arranger les composants dans cinq zones: **north**, **south**, **east**, **west** et **center**
- Disposition **par défaut** de **JFrame** et **JDialog**

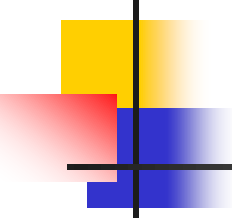




FlowLayout

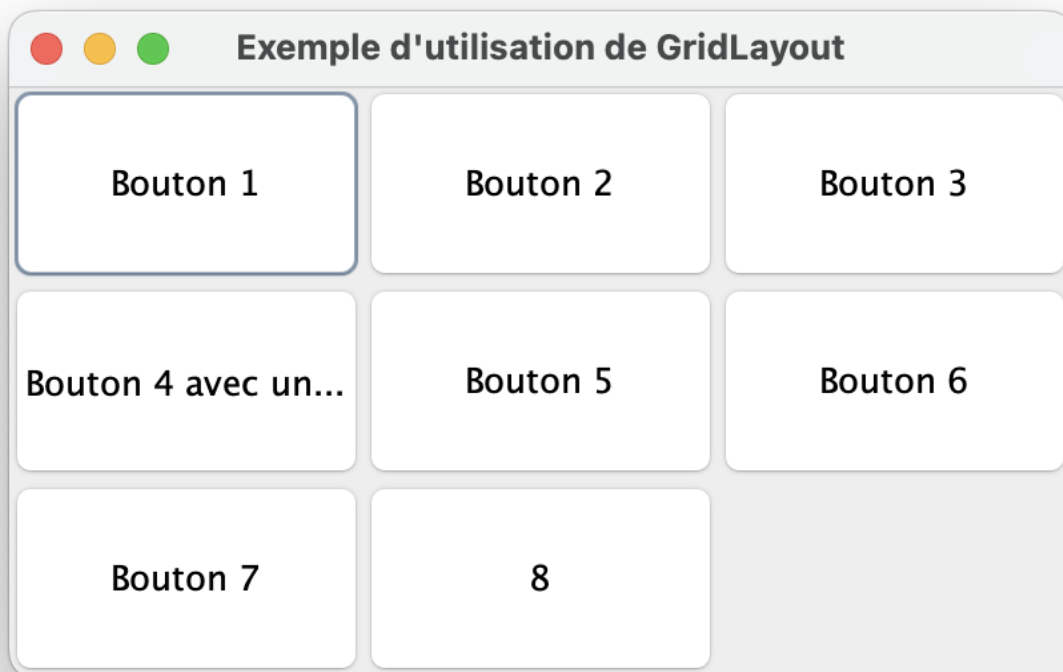
- **FlowLayout** permet de placer les composants ligne par ligne, les uns après les autres
- Disposition **par défaut** de **JPanel**





GridLayout

- **GridLayout** permet de placer les composants sur une grille rectangulaire
- Le conteneur est divisé en cellules de même taille

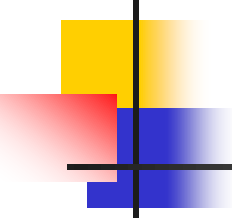


GridLayout

■ Exemple

```
public class TestGridLayout{
    public static void main(String[] args) {
        JFrame fen = new JFrame("Test de GridLayout");
        JLabel label = new JLabel("Label d'affichage au centre", JLabel.CENTER);
        JPanel panel = new JPanel();
        JButton btn1 = new JButton("Valider");
        JButton btn2 = new JButton("Annuler");
        panel.add(btn1);
        panel.add(btn2);
        fen.setLayout(new GridLayout(2, 1));
        fen.add(label);
        fen.add(panel);
        fen.setSize(300, 200);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setVisible(true);
    }
}
```



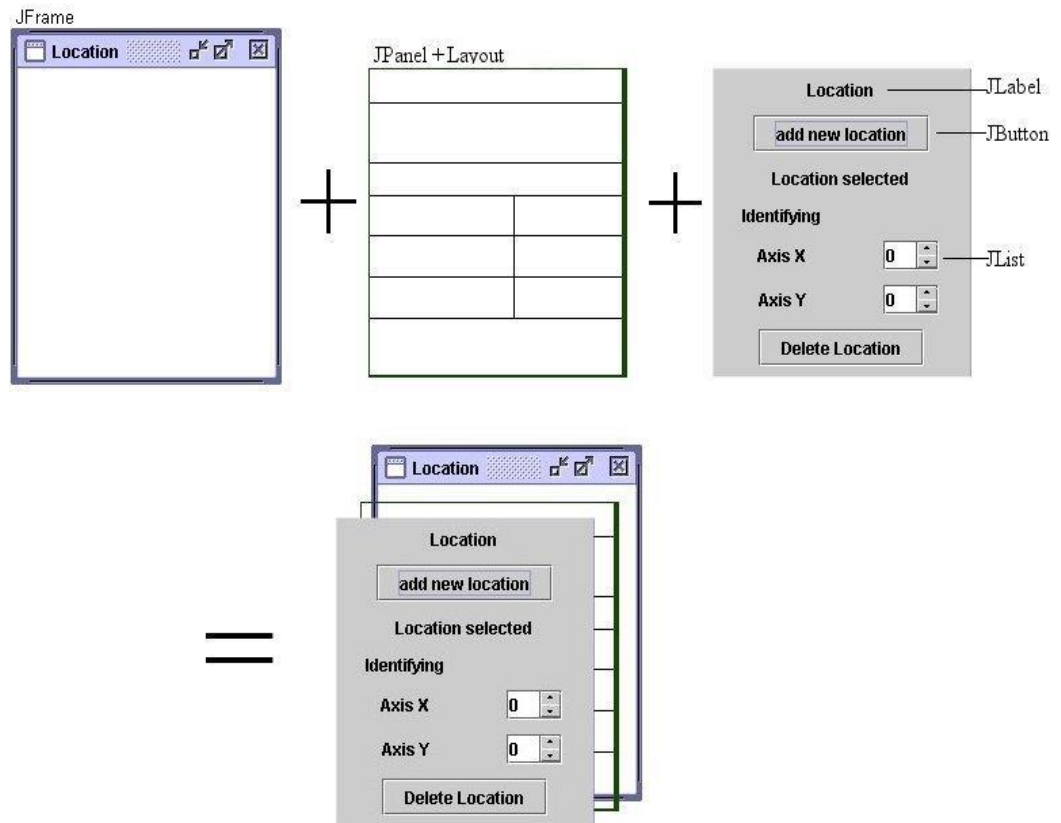


CardLayout

- **CardLayout** gère les composants en sorte qu'un seul composant soit visible à la fois
- Chaque composant est considérée comme une carte
- Il permet de limiter l'utilisation de multiple fenêtres
- Méthodes couramment utilisées
 - **next(Container)**: retourner à la prochaine carte du conteneur
 - **previous(Container)**: retourner à la carte précédente du conteneur
 - **first(Container)**: retourner à la première carte du conteneur
 - **last(Container)**: retourner à la dernière carte du conteneur
 - **show(Container, String)**: retourner à la carte spécifiée avec le nom donné

Etapes de conception d'une fenêtre

- Création de la fenêtre (**JFrame**)
- Intégration de panneaux (**JPanel**)
- Utilisation d'un **Layout** pour organiser les composants d'un panneau
- Intégration des composants dans les panneaux (**JPanel**)



Exercice d'application

- Écrire un programme Java qui permet de reproduire l'interface suivante:

Formulaire d'inscription

Prénoms

Nom

Sexe ☐ Homme ☐ Femme

Profession 

Adresse

Valider Annuler